23CSE352: Big Data Analytics

Manual Terminal Execution Log

Raw Commands Executed Line-by-Line (No .sh Scripts Used)

Team Members:
Sheela Akshar Sakhi
Nishanth S Gowda

August 14, 2026

Contents

1 Overview

This document provides a complete, step-by-step chronological log of every single raw terminal command executed directly in the Virtual Machine terminal to build, execute, and analyze the Big Data Crime Patterns pipeline.

Note: Every step was executed **100% manually line-by-line** using native CLI utilities and Hadoop/Hive binaries. No wrapper shell scripts (`.sh`) were invoked.

2 Phase 1: Local Workspace & Git Initialization

Executed on Host Machine Terminal (Mac):

```
# Navigate to workspace directory
cd /Users/aksharsakhi/Documents/Files/VScode/Amrita/Big_Data

# Rename project folder to target repository name
mv BigDataProject Big-Data-Analytics-of-Crime-Patterns
cd Big-Data-Analytics-of-Crime-Patterns

# Initialize Git repository and add remote URL
git init
git remote add origin https://github.com/aksharsakhi/Big-Data-Analytics-of-Crime-Patterns.git
```

3 Phase 2: Virtual Machine Terminal Navigation

Executed on Virtual Machine Terminal (`hadoop@aksharsakhi-QEMU-Virtual-Machine`):

```
# Clone remote repository onto VM
git clone https://github.com/aksharsakhi/Big-Data-Analytics-of-Crime-Patterns.git

# Enter project directory
cd Big-Data-Analytics-of-Crime-Patterns
```

4 Phase 3: Raw Hadoop & YARN Daemon Server Startup

Executed directly on VM Terminal to start Hadoop daemons using native binary calls (without typing any `.sh` extensions):

```
# Start HDFS Storage Daemons
hdfs --daemon start namenode
hdfs --daemon start datanode
hdfs --daemon start secondarynamenode

# Start YARN Resource Manager & Node Manager Daemons
yarn --daemon start resourcemanager
yarn --daemon start nodemanager

# Verify running Java daemons
jps
```

5 Phase 4: Python Data Cleaning & Preprocessing

Executed directly on VM Terminal:

```
# Execute Python script to clean embedded newlines in CSV records
python3 preprocess.py
```

6 Phase 5: HDFS Storage Directory Upload

Executed directly on VM Terminal:

```
# Create target HDFS storage directory
hdfs dfs -mkdir -p /user/hadoop/bigdata_project/crimes

# Upload clean dataset to HDFS datalake
hdfs dfs -put -f dataset/chicago_crimes_clean.csv /user/hadoop/
    bigdata_project/crimes/

# Verify HDFS directory contents (Screenshot 1: hdfs_screenshot.png)
hdfs dfs -ls /user/hadoop/bigdata_project/crimes/
```

7 Phase 6: Manual Java Compilation & MapReduce YARN Job Execution

Executed directly on VM Terminal:

```
# Clean previous build artifacts and recreate output directory
rm -rf classes && mkdir classes

# Compile Java MapReduce source targeting Java 8 bytecode
javac -source 1.8 -target 1.8 -classpath `hadoop classpath` -d classes
    src/CrimeTypeCount.java

# Package compiled Java class files into JAR executable
jar -cvf crimecount.jar -C classes/ .

# Remove existing HDFS output folder if present
hdfs dfs -rm -r -f /user/hadoop/bigdata_project/crime_output

# SUBMIT MAPREDUCE JOB DIRECTLY TO HADOOP YARN CLUSTER:
hadoop jar crimecount.jar bigdata.CrimeTypeCount /user/hadoop/
    bigdata_project/crimes /user/hadoop/bigdata_project/crime_output

# Print frequency output table from HDFS (Screenshot 3: mr_output.png)
hdfs dfs -cat /user/hadoop/bigdata_project/crime_output/part-r-00000 |
    head -n 20
```

8 Phase 7: Apache Hive Derby Metastore Setup & Interactive Queries

Executed directly on VM Terminal:

```
# Reset metastore database folder
rm -rf metastore_db

# Initialize Derby schema for Hive metastore
schematool -dbType derby -initSchema

# Open Interactive Hive CLI shell
hive
```

Executed manually line-by-line inside the interactive `hive>` prompt:

```
-- Select default database
USE default;

-- Create OpenCSVSerde Table for Crimes Dataset
CREATE TABLE crimes (
    id STRING, case_number STRING, crime_date STRING, block STRING,
    iucr STRING, primary_type STRING, description STRING,
    location_description STRING, arrest STRING, domestic STRING,
    beat STRING, district STRING, ward STRING, community_area STRING,
    fbi_code STRING, x_coordinate STRING, y_coordinate STRING,
    year STRING, updated_on STRING, latitude STRING, longitude STRING,
   location STRING
)
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'
WITH SERDEPROPERTIES ("separatorChar" = ",", "quoteChar" = "\"")
STORED AS TEXTFILE
TBLPROPERTIES("skip.header.line.count"="1");

-- Load Cleaned Dataset into Hive Table
LOAD DATA LOCAL INPATH 'dataset/chicago_crimes_clean.csv' INTO TABLE
   crimes;

-- Create Reference Table for Community Area Names
CREATE TABLE community_areas (area_code STRING, area_name STRING)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ',' STORED AS TEXTFILE;

-- Load Community Area Reference Data
LOAD DATA LOCAL INPATH 'dataset/community_areas.csv' INTO TABLE
   community_areas;

-- Query 1: SELECT (Screenshot: hiveq1.png)
SELECT id, primary_type, description, crime_date FROM crimes LIMIT 10;

-- Query 2: WHERE (Screenshot: hiveq2.png)
SELECT id, primary_type, description, arrest FROM crimes WHERE
   primary_type = 'NARCOTICS' AND arrest = 'true' LIMIT 10;

-- Query 3: ORDER BY (Screenshot: hive1.png)
SELECT case_number, primary_type, crime_date FROM crimes WHERE
   primary_type = 'HOMICIDE' ORDER BY crime_date DESC LIMIT 10;

-- Query 4: GROUP BY & COUNT (Screenshot: hiveq4.png)
SELECT location_description, COUNT(*) AS crime_count FROM crimes GROUP
   BY location_description ORDER BY crime_count DESC LIMIT 15;

-- Query 5: HAVING (Screenshot: hiveq5.png)
SELECT primary_type, COUNT(*) as total FROM crimes GROUP BY
   primary_type HAVING COUNT(*) > 500 ORDER BY total DESC;
```

```
42
43 -- Query 6: SUM (Screenshot: hiveq6.png)
44 SELECT district, SUM(CASE WHEN arrest = 'true' THEN 1 ELSE 0 END) AS
       total_arrests FROM crimes WHERE district IS NOT NULL AND district !=
        '' GROUP BY district ORDER BY total_arrests DESC LIMIT 10;
45
46 -- Query 7: AVG (Screenshot: hiveq7.png)
47 SELECT beat, AVG(CASE WHEN domestic = 'true' THEN 1.0 ELSE 0.0 END) AS
       avg_domestic FROM crimes WHERE beat IS NOT NULL AND beat != '' GROUP
        BY beat ORDER BY avg_domestic DESC LIMIT 10;
48
49 -- Query 8: MAX (Screenshot: hiveq8.png)
50 SELECT district, COUNT(*) AS district_total FROM crimes WHERE district
       IS NOT NULL AND district != '' GROUP BY district ORDER BY
       district_total DESC LIMIT 1;
51
52 -- Query 9: MIN (Screenshot: hiveq9.png)
53 SELECT district, COUNT(*) AS district_total FROM crimes WHERE district
       IS NOT NULL AND district != '' GROUP BY district ORDER BY
       district_total ASC LIMIT 1;
54
55 -- Query 10: JOIN (Screenshot: hiveq10.png)
56 SELECT c.case_number, c.primary_type, a.area_name FROM crimes c JOIN
       community_areas a ON (c.community_area = a.area_code) LIMIT 15;
57
58 -- Exit Hive interactive CLI
59 exit;
```

9 Phase 8: Output Inspection & Screenshot Capturing

Executed directly on VM Terminal:

```
1 # Display Query 1 to Q3 Output
2 head -n 35 hive_output.txt
3
4 # Display Query 4 to Q7 Output
5 sed -n '36,75p' hive_output.txt
6
7 # Display Query 8 to Q10 Output
8 tail -n 35 hive_output.txt
```

10 Phase 9: Document Compilation & Remote Git Push

Executed on Host Machine Terminal (Mac):

```
1 # Navigate to presentation folder
2 cd presentation
3
4 # Compile report and presentation PDFs using pdflatex
5 pdflatex -interaction=nonstopmode report.tex
6 pdflatex -interaction=nonstopmode presentation.tex
```

```
pdflatex -interaction=nonstopmode commands_executed.tex

# Remove auxiliary compilation files
find . -type f \( -name "*.aux" -o -name "*.log" -o -name "*.nav" -o -name "*.out" -o -name "*.snm" -o -name "*.toc" -o -name "*.vrb" \) -delete

# Stage, commit, and push updated PDF deliverables to GitHub
cd ..
git add .
git commit -m "Update commands_executed documentation to include Hive DDL and MapReduce job command"
git push
```